



Programación de Estructuras de Datos y Algoritmos Fundamentales

Actividad Integradora 5 Estructuras de datos jerárquicas

Farid Gabriel Velasco Martínez A01736669

6 de Junio de 2025

Índice

Índice	2
Introducción	3
Beneficios del heap	3
Análisis de la complejidad computacional de las operaciones básicas del heap	4
Pseudocódigo del algoritmo de búsqueda del mínimo elemento con condición	5
Conclusión	6
Referencias:	6

Introducción

En esta última actividad integradora desarrollamos una aplicación auxiliar para la consulta de datos críticos sobre una IP específica, para ello hicimos uso de una implementación eficiente de una tabla hash de hashing cerrado y con la técnica quadratic probing para manejar las colisiones. Esta aplicación puede usarse en conjunto con la desarrollada en la actividad integradora 4 para analizar y obtener información de manera eficiente de una serie de datos como pueden ser las IP y sus interconexiones, datos a los que tenemos acceso.

Esta nueva estructura de datos, de dispersión, es especialmente útil en muchos escenarios, especialmente aquellos en los que la eficiencia temporal sea la prioridad, una tabla hash bien diseñada e implementada tiene una complejidad temporal de sus métodos principales como `insert()`, `remove()`, `find()`, etc., de $O(1)$ en promedio. Aunque su diseño y estructura sacrifican eficiencia espacial por eficiencia temporal, es común que se ocupen en aplicaciones o sistemas de alta carga computacional donde los componentes de hardware que los componen tienen las características suficientes como para que este sacrificio no sea notable.

Beneficios de las Tablas Hash

Las Tablas Hash se especializan en guardado y acceso de información, incorporan técnicas como la codificación de llaves (función hash) y quadratic probing para que métodos cruciales se ejecuten en tiempo constante $O(1)$ en promedio. Si bien carecen de mecanismos de reorganización de sus elementos, como algoritmos específicos de ordenamiento, son usadas en conjunto con otras estructuras de datos para ya sea constituir estructuras de datos complejas o bien hacer más eficiente temporalmente la implementación de ciertos algoritmos y/o estructuras de datos.

En los grafos por ejemplo, el uso de Tablas Hash es crucial, ya que se realizan una gran cantidad de accesos a los elementos de un grafo en sus operaciones, y tener la capacidad de acceder a ellos en tiempo constante es extremadamente valioso, de otra forma, si se usaran vectores, árboles o listas, de no ser por un diseño que sacrifique funcionalidad por eficiencia, la complejidad temporal mínima que se podría alcanzar es $O(\log n)$ en acceso a elementos, ahora bien arrastrar esta complejidad a la que tienen los métodos de un grafo hace que este pierda robustez y eficiencia en general.

Las Tablas Hash también son útiles fuera de ser componentes de otras estructuras de datos, pueden utilizarse en sistemas generales que manejen datos o archivos para indexado rápido y búsqueda rápida.

Complejidad temporal de los métodos principales de las Tablas Hash

Los métodos principales en las Tablas Hash son:

Insert: $O(1)$ Remove: $O(1)$ Find: $O(1)$

Estos métodos se basan en una función hash que convierte la llave del elemento a guardar en un valor numérico traducido al índice en un vector, de esta forma con la llave podemos localizar de manera rápida un elemento a partir de su llave. No podemos decir sin embargo, que cada llave tiene un índice único, es probable que 2 llaves tras la función hash obtengan el mismo índice, lo cual se conoce como colisión, en este caso usamos la técnica de Quadratic Probing :

$$index = (hash(key) + i^2) \bmod capacity$$

Para manejar estas colisiones.

La incidencia de colisiones depende valor de alfa (factor de carga) cuya relación se puede expresar como la siguiente función aproximadamente:

$$C(\alpha) = \frac{1}{1-\alpha} - 1$$

No se puede garantizar que a valores bajos de alfa no existan colisiones, aunque que a valores de alfa menores incluso a 0.7 el rendimiento es aceptable, sin embargo pasado ese umbral el crecimiento de las colisiones es indeseable. En ese caso los métodos de las Tablas Hash se pueden llegar a degradar incluso hasta $O(n)$ o en el peor de los casos, para Insert(), que se genere un bucle del Quadratic Probing en el cual si la estructura no está bien diseñada, se llegue al nodo original sin haber pasado por todos los índices del vector de almacenamiento, dejando potencialmente espacios vacíos a los que cada vez es más difícil llegar mientras que esta llave que entró en un bucle no se podrá almacenar.

Se graficó el comportamiento de las colisiones en función de alfa en 2 escenarios distintos:

Datos constantes:

- 5 mil entradas
- Llaves numéricas aleatorias entre 0 y 20000
- Data es una string sencilla

Diferenciador:

- En el primer caso se configuró la restricción de insert para que hiciera rehash una vez que el tamaño de la tabla fuera igual a su capacidad
- En el segundo caso se configuró la restricción a modo que a un valor igual o mayor que 0.75 de alfa, se aplique rehashing

Resultados:

Caso 1:

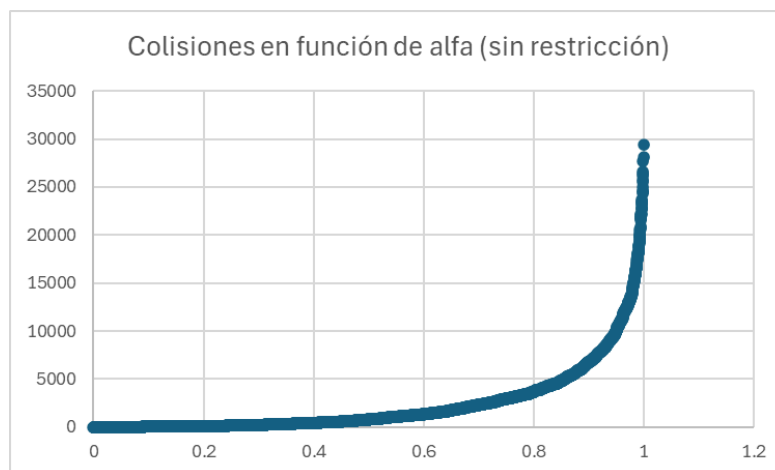


Figura 1. Gráfica de colisiones como función de alfa sin restricción de alfa

Caso 2:

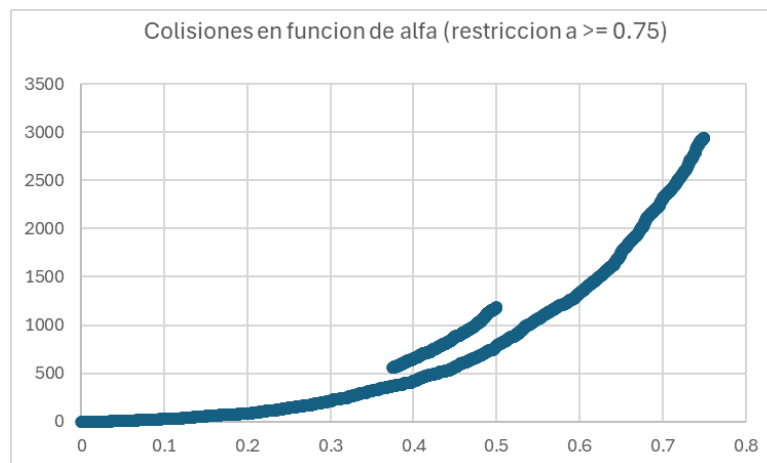


Figura 2. Gráfica de colisiones como función de alfa con restricción en alfa ≥ 0.75

De las anteriores gráficas podemos concluir visualmente de manera rápida que la restricción de alfa tiene un impacto significativo en la cantidad de colisiones obtenidas, el punto más alto en la Figura 1 corresponde aproximadamente a >29000 colisiones, mientras que el punto más alto en la Figura 2 corresponde aproximadamente a >2900 colisiones, es una orden de magnitud de diferencia que en la práctica hace mucha diferencia

Al repetir el experimento reduciendo la capacidad de la tabla a mil elementos obtenemos gráficas idénticas:

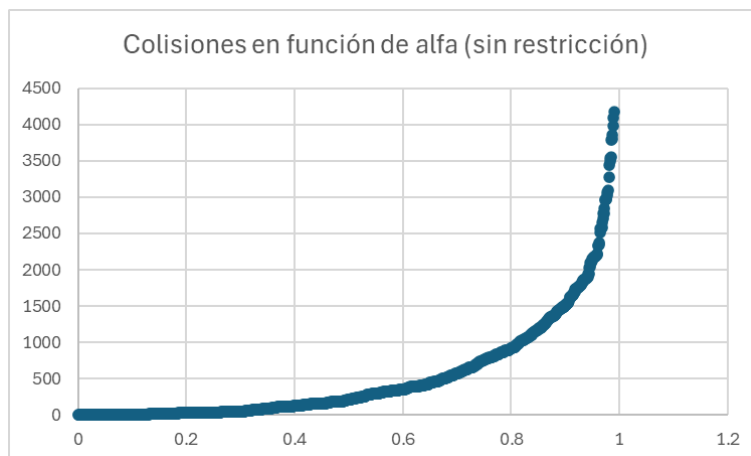


Figura 3. Gráfica de colisiones como función de alfa sin restricción de alfa y capacidad máxima de la tabla de mil elementos.

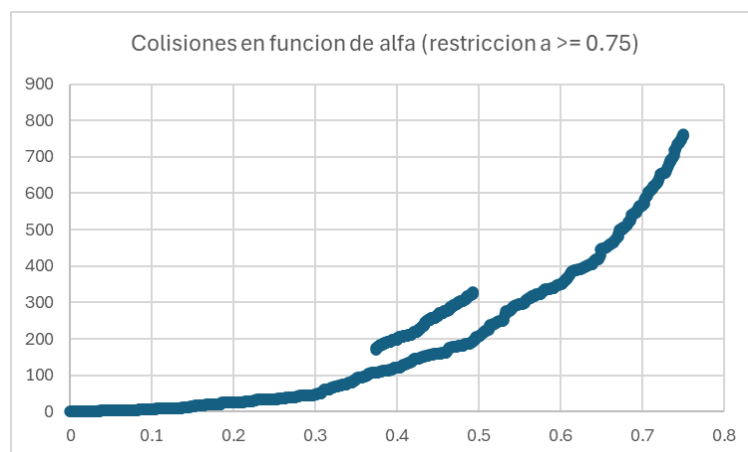


Figura 4. Gráfica de colisiones como función de alfa con restricción en $\alpha \geq 0.75$ y capacidad máxima de la tabla de mil elementos.

Algo que podemos distinguir sin embargo, es que entre más cercano es el valor de alfa a 1, las colisiones están mucho más dispersas en el eje. Esto es consistente con el modelo matemático propuesto anteriormente ya que el valor de Colisiones a un valor de alfa infinitamente cercano a 1 también tenderá al infinito.

IP participante en ataque DDOS

De los casos de prueba proporcionados, se obtuvo que la IP 73.89.221.25 y la IP 185.109.34.183 tienen un total de 18 IPs accedidas. En la anterior actividad integradora se consideró que la IP que haya comenzado sus actividades antes en el tiempo es la que tiene más probabilidades de ser el origen de un ataque DDOS, por lo que se presume que la IP 185.109.34.183 es la dirección donde se encuentra el ordenador que está causando el ataque DDOS, por otro lado tiene una menor cantidad de accesos entrantes (6) que la IP 73.89.221.25 (7), por lo que una IP con menor ratio de accesos entrantes y salientes es más probable que tenga un comportamiento natural o menos sospechoso, a diferencia de una con alto ratio entre los tipos de accesos.

Referencias:

1. *Acelera tus operaciones con IOT*. (s. f.).
<https://www.oracle.com/mx/internet-of-things/>
2. Condliffe, J. (2020, 2 abril). The FBI Shut Down a Huge Botnet, but There Are Plenty More Left. *MIT Technology Review*.
<https://www.technologyreview.com/2017/04/11/152645/the-fbi-shut-down-a-huge-bot-net-but-there-are-plenty-more-left/>
3. GeeksforGeeks. (2022, 1 septiembre). *Minimum element in a max heap*.
GeeksforGeeks. <https://www.geeksforgeeks.org/minimum-element-in-a-max-heap/>
4. GeeksforGeeks. (2023, 10 octubre). *Insertion and Deletion in Heaps*. GeeksforGeeks.
<https://www.geeksforgeeks.org/insertion-and-deletion-in-heaps/>
5. GeeksforGeeks. (2025a, enero 2). *Heap Sort Data Structures and Algorithms Tutorials*. GeeksforGeeks. <https://www.geeksforgeeks.org/heap-sort/>

6. GeeksforGeeks. (2025b, marzo 19). *Priority Queue in C++ STL*. GeeksforGeeks.
<https://www.geeksforgeeks.org/priority-queue-in-cpp-stl/>
7. GeeksforGeeks. (2025c, marzo 21). *Building Heap from Array*. GeeksforGeeks.
<https://www.geeksforgeeks.org/building-heap-from-array/>
8. Inside code. (2021, 28 agosto). *Heaps, heapsort, and priority queues - Inside code* [Video]. YouTube. <https://www.youtube.com/watch?v=pLIajuc31qk>
9. Spanning Tree. (2020, 16 julio). *What Is a Binary Heap?* [Video]. YouTube.
<https://www.youtube.com/watch?v=AE5I0xACpZs>
10. Robinson, S., Hanna, K. T., & Lutkevich, B. (2025, 18 febrero). *What is a botnet?* Search Security.
https://www.techtarget.com/searchsecurity/definition/botnet?track=NL-1823&ad=931942&src=931942&asrc=EM_NLN_122529935